

A PARAMETRIC, RESOURCE-BOUNDED GENERALIZATION OF LÖB'S THEOREM, AND A ROBUST COOPERATION CRITERION FOR OPEN-SOURCE GAME THEORY

ANDREW CRITCH

Abstract. This paper presents two theorems: (1) a generalization of Löb's Theorem that applies to formal proof systems operating with bounded computational resources, such as formal verification software or theorem provers, and (2) a theorem on the robust cooperation of agents that employ proofs about one another's source code as unexploitable criteria for cooperation. The latter illustrates a capacity for out-performing classical Nash equilibria and correlated equilibria, attaining mutually cooperative program equilibrium in the Prisoner's Dilemma while remaining unexploitable, i.e., sometimes achieving the outcome (Cooperate, Cooperate), and never receiving the outcome (Cooperate, Defect) as player 1.

§1. Introduction. In the game theoretic analysis of computerized agents who may read one another's source code, and more generally in the analysis of program verification systems that use proofs to verify other program verification systems, a need has arisen for a version of Löb's theorem that applies to proof systems with bounded computational resources. The first main theorem of this paper is a generalization of Löb's Theorem that applies in such cases, developed and proven in Section 2 through Section 4.

The second main theorem, presented in Section 5, makes use of the first theorem and some further proof-theoretic analysis to derive some implications for the game theory of agents who read one another's source code. Specifically, game theoretic work of Bárász, LaVictoire, Christiano et al. [2014] and LaVictoire, Fallenstein, Yudkowsky et al. [2014] found that Löb's Theorem can be used to design entities in modal logic that resemble "agents" who achieve robust cooperative equilibria in games such as the Prisoner's Dilemma. However, these so-called "modal agents" were defined as strings representing uncomputable functions; strings of the form "if $(\dots)$ is provable, return 1, else return 0". It remained open whether their results would arise naturally for computable agents, a question answered affirmatively in Section 5.

1.1. Related work. The work of Pudlák [1998] on the lengths of proofs of finitistic consistency statements are suggestive of the first theorem of this paper. Specifically, Pudlák shows that, for a consistent system, there exists some $\varepsilon > 0$ such that for all N , any proof of a statement of the form "there does not exist

a proof of $\perp$ using N or fewer characters” must use at least N^ε characters. This means there is some obstruction to self-trust for a resource-bounded proof system, which suggests that a resource-bounded version of Löb’s theorem—a generic statement of self-trust about a family of sentences parametrized by a resource bound—might also hold.

§2. Fundamentals. Since this paper draws from work in several disciplines, this section is provided to clarify the use of notation and conventions throughout.

2.1. Proof length conventions and notation. Proof length will be measured in *characters* instead of lines, the way one might measure the size of a text file on a computer. An extensive analysis of proof lengths measured in characters is covered by [?].

Throughout this paper, S refers to a fixed proof system (e.g. an extension of Peano Arithmetic). Writing

$$S \vdash_n \phi, \quad \text{or simply} \quad \vdash_n \phi$$

means that there exists an S -proof of ϕ using n or fewer characters.

2.2. Proof system. Let S be any first-order proof system that

- 1) can represent computable functions (e.g., by being an extension of $\mathcal{PA}$; see Section 2.4),
- 2) can write any number $k \in \mathbb{N}$ using $\mathcal{O}(\lg(k))$ symbols (e.g. using binary), and
- 3) allows the definition and use of abbreviations during proofs (see the Appendix for details).

Compact numeral representations and abbreviations are allowed in the proof system for two reasons. The first is that real-world automated proof systems will tend to use these because of memory constraints. The second is that abbreviations make the lengths of shortest-proofs slightly easier to analyze. For example, if a number N with a very large number of digits occurs in the shortest proof of a proposition, it will not occur multiple times; instead, it will occur only once, in the definition of an abbreviation for it. Then, one does not need to carefully count the number of times the numeral occurs in the proof to determine the contribution of its size to the proof length; the contribution will simply be linear in its length, or $\lg(N)$.

Write

L_S for the language of S ,
 $L_S(r)$ for the set of formulas in L_S with r free variables, and
 $\text{Const}(S)$ for the set of closed-form constant expressions in S (e.g. 0 , $S0$, $S0 + S0$, etc.).

When $\phi \in L_S(r)$, given any closed-form expressions $c_1, \dots, c_r$ (such as constants, or free variables), write

$$\phi[\vec{c}] = \phi[c_1, \dots, c_r]$$

for the result of substituting the c_i for the free variables in ϕ .

2.3. Choosing a Gödel encoding. Along with the proof system S , a single encoding

$$\#(-) : L_S \rightarrow \mathbb{N}$$

is chosen and fixed throughout, as well as a “numeral” mapping

$$^\circ(-) : \mathbb{N} \rightarrow \text{Const}(S) \subseteq L_S$$

for expressing naturals as constants in S . Note that in traditional $\mathcal{PA}$, for example, $^\circ 5 = \mathcal{SSSSS}0$. However, to be more realistic it is assumed that S uses a binary encoding to be more efficient, so e.g., $^\circ 5 = 101$. The maps $\#(-)$ and $^\circ(-)$ combine to form a Gödel encoding

$$\ulcorner(-)\urcorner : L_S \rightarrow \text{Const}(S)$$

$$\ulcorner\phi\urcorner := ^\circ\#\phi$$

which allows S to write proofs about itself.

2.4. Representing computable functions. It is assumed that for any computable function $f : \mathbb{N} \rightarrow \mathbb{N}$, there exists a “graph” formula $\Gamma_f \in L_S(2)$ such that

$$\text{for all } x \in \mathbb{N}, S \vdash (\forall y) (\Gamma_f[^\circ x, y] \leftrightarrow y = ^\circ f(x)).$$

For example, this condition holds if S is an extension of $\mathcal{PA}$; see, e.g. Theorem 6.8 of Cori and Lascar [2001, Part II].

Sometimes we abuse notation and write symbols for computable functions inline with logical expressions to save space. For example, given functions f, g and h , to say that S proves that for any x value, $f(x) < g(x) + h(x)$, technically one should write

$$\vdash (\forall x)(\forall y_1)(\forall y_2)(\forall y_3) (\Gamma_f[x, y_1] \text{ and } \Gamma_g[x, y_2] \text{ and } \Gamma_h[x, y_3] \rightarrow y_1 < y_2 + y_3)$$

but instead, we abuse notation and write

$$\vdash (\forall x)(f(x) < g(x) + h(x)).$$

2.5. Asymptotic notation. The notation $f \prec g$ will mean that for any $M \in \mathbb{N}$, there exists an $N \in \mathbb{N}$ such that for all $n > N$, $Mf(n) < g(n)$. The expression $\mathcal{O}(g)$ stands for the set of functions $f \preceq g$. If $f : \mathbb{N} \rightarrow \mathbb{N}$ is any specific function, $f(\mathcal{O}(g))$ will stand for the set of functions of the form $e \circ f$ where $e \in \mathcal{O}(g)$.

§3. A Bounded Provability Predicate, $\Box_k$. Here a predicate $\Box_k$ is defined for asserting provability using a proof with length bounded by k .

3.1. Defining $\Box_k$. Given a choice of Gödel encoding for Peano Arithmetic, it is classical that a formula $Bew[m, n] \in L_S(2)$ exists that means, in natural language, that the number m encodes a proof in $\mathcal{PA}$, and that the number n encodes the statement it proves. So, the standard provability operator $\Box : L_{\mathcal{PA}} \rightarrow L_{\mathcal{PA}}$ can be defined as

$$\Box(\phi) := (\exists m)(Bew[m, \ulcorner\phi\urcorner]).$$

It is taken for granted that $Bew[m, n]$ exists for S and can be extended to a “bounded Bew” formula $BBew[m, n, k] \in L_S(3)$ that means

- m encodes a proof in S ,
- n encodes the statement it proves, and
- the proof encoded by m , before being encoded as m , uses at most k characters when written in the language of S . (Note that in general, m itself will be much larger than k as a member of $\mathbb{N}$.)

Then one can define a “bounded” box operator, which, given any sentence ϕ and closed expression k (such as a free variable, or a constant), returns

$$\Box_k(\phi) := (\exists m)(BBew[m, \ulcorner \phi \urcorner, k]).$$

Also taken for granted is a computable “single variable evaluation” function, $Eval_1 : \mathbb{N} \rightarrow \mathbb{N}$, such that for any $\phi \in L_S(1)$,

$$Eval_1(\ulcorner \phi \urcorner, k) := \ulcorner \phi(\circ k) \urcorner.$$

Since $Eval_1$ is computable, it can be represented in L_S as in Section 2.4. This allows us to extend the $\Box_k$ operator to act on sentences with one unbound variable. Specifically, if ϕ is a sentence with an unbound variable ℓ , then

$$\Box_k(\phi) := (\exists m)(BBew[m, Eval_1(\ulcorner \phi \urcorner, \ell), k]).$$

Then $\Box_k(\phi)$ itself has ℓ as an unbound variable, and in natural language stands for “There is a proof using k or fewer characters of the formula ϕ ”.

3.2. Basic properties of $\Box_k$. Each of the following properties will be needed multiple times during the proof of the main results. Since the proof is already highly symbolic, these properties are given English names to recall them.

PROPERTY 1 (Implication Distribution). *There is a constant $c \in \text{Const}(S)$ such that for any $p, q \in L_S$,*

$$\vdash (\forall a)(\forall b) \left(\Box_a(p \rightarrow q) \rightarrow \left(\Box_b(p) \rightarrow \Box_{a+b+c}(q) \right) \right).$$

PROOF SKETCH. The fact that one can combine a proof of an implication with the proof of its antecedent to obtain a proof of its consequent can be proven in general, with quantified variables in place of the Gödel numbers of the particular statements involved. Let us suppose this general proof has length c_0 . Then, one needs only to instantiate the statements in it to p and q . However, if p and q are long expressions, they can have been abbreviated in the earlier proofs without lengthening them, so they can be written in abbreviated form again during this step. Hence, the total cost of combining the two proofs is around $c = 2c_0$, which is constant with respect to p and q . $\dashv$

PROPERTY 2 (Quantifier Distribution). *There is a constant $C \in \text{Const}(S)$ such that for any $\phi \in L_S(1)$,*

$$\begin{aligned} & \vdash \Box_N \left((\forall k)(\phi[k]) \right) \\ \Rightarrow & \vdash (\forall k) \left(\Box_{C+2N+\lg(k)}(\phi[k]) \right), \text{ which in turn} \\ \Rightarrow & \vdash (\forall k) \left(\Box_{\mathcal{O}(\lg(k))}(\phi[k]) \right). \end{aligned}$$

In the final subscript, $\mathcal{O}(\lg(k))$ stands for a closed expression representing a function of k that is asymptotically less than some positive constant times $\mathcal{O}(\lg(k))$.

PROOF. An encoded proof of $\phi[{}^\circ K]$ for a specific K can be obtained by specializing the conclusion of an N -character encoded proof of $(\forall k)(\phi[k])$ and appending the specialization with ${}^\circ K$ in place of k at the end. To avoid repeating ${}^\circ K$ numerous times in the final line (in case it is large), an abbreviation will be used for ϕ . Thus the appended lines can say:

- (1) let Φ stand for $\ulcorner \phi \urcorner$
- (2) $\Phi[{}^\circ K]$

Let us analyze how many characters are needed to write such lines. First, a string Φ is needed to use as an abbreviation for ϕ . Since no string of length $\frac{N}{2}$ has yet been used as an abbreviation in the earlier proof (otherwise one can shorten the proof by not defining and using the abbreviation), one can achieve $\text{Length}(\Phi) < \frac{N}{2}$. As well, some constant c number of characters are needed to write out the system's equivalent of “let”, “stand for”, “(”, and “)”. Finally, $\lg(K)$ characters are needed to write ${}^\circ K$. Altogether, the proof was extended by $C + N + \lg(k)$ characters, for a total length of $2N + c + \lg(k)$. $\dashv$

§4. A bounded generalization of Löb's theorem. This section exhibits the first main theorem: a generalization of Löb's theorem applicable to the analysis of resource-bounded proof systems.

DEFINITION 1 (Proof expansion function). Let

$$e : \mathbb{N} \rightarrow \mathbb{N}$$

be any computable function bounding the expansion of S -proof lengths when they are Gödel-encoded. That is, its definition is only that it must be large enough to satisfy the following two properties:

PROPERTY 3 (Bounded Necessitation). For all $\phi \in L_S$,

$$\vdash_k \phi \tag{4.1}$$

$$\Rightarrow \vdash_{e(k)} \Box_k(\phi) \tag{4.2}$$

PROPERTY 4 (Bounded Inner Necessitation). For any $\phi \in L_S$,

$$\vdash \Box_k(\phi) \rightarrow \Box_{e(k)}(\Box_k(\phi)).$$

Then the following theorem holds:

THEOREM 1 (Resource-bounded generalization of Löb's theorem). Let $p[k] \in L_S(1)$ be a formula with a single unquantified variable k , and suppose that $f : \mathbb{N} \rightarrow \mathbb{N}$ is computable and satisfies $f(k) \succ e(\mathcal{O}(\lg(k)))$. Then there is a threshold $\hat{k} \in \mathbb{N}$, depending on $p[k]$, such that

$$\begin{aligned} & \vdash (\forall k) \left(\Box_{f(k)}(p[k]) \rightarrow p[k] \right) \\ \Rightarrow & \vdash (\forall k > \hat{k}) (p[k]) \end{aligned}$$

Note: In fact a weaker statement,

$$\vdash (\forall k > k_1) \left(\Box_{f(k)}(p[k]) \rightarrow p[k] \right),$$

is sufficient to derive the consequent, since we could just redefine $f(k)$ to be 0 for $k \leq k_1$ and then $\Box_{f(k)}(p[k]) \rightarrow p[k]$ is vacuously true and provable for $k \leq k_1$ as well.

The proof of Theorem 1 makes use of a version of Gödel's diagonal lemma that allows free variables to appear in the formula being diagonalized:

PROPOSITION 1. *Suppose S is a first-order theory capable of representing all computable functions, as in Section 2.4. Then for any formula $G \in L_S(r+1)$, there exists a formula $\psi \in L_S(r)$ such that:*

$$\vdash (\forall \bar{k}) (\psi[\bar{k}] \leftrightarrow G[\ulcorner \psi \urcorner, \bar{k}]),$$

where $\bar{k} = (k_1, \dots, k_r)$ are free variables.

PROOF. This result can be found on p. 53 of Boolos [1993]. $\dashv$

PROOF OF THEOREM 1. *(In this proof, each centered equation will follow directly from the one above it unless otherwise noted.)*

We begin by choosing some function $g(k)$ such that $\lg(k) \prec g(k)$ and $e(g(k)) \prec f(k)$. For example, we could take $g(k) = \lfloor \sqrt{(\lg(k))(e^{-1}(f(k)))} \rfloor$. Define a formula $G \in L_S(2)$ by

$$G[n, k] := ((\exists m)(Bew[m, Eval_1(n, k), g(k)])) \rightarrow p[k]$$

so that for any $\phi \in L_S(1)$,

$$G[\ulcorner \phi \urcorner, k] = \Box_{g(k)}(\phi[k]) \rightarrow p[k].$$

Now, by Proposition 1, there is some $\psi \in L_S(1)$ such that in some number of characters n ,

$$\vdash_n (\forall k) (\psi[k] \leftrightarrow G[\ulcorner \psi \urcorner, k]). \quad (4.3)$$

By Bounded Necessitation,

$$\vdash \Box_n ((\forall k) (\psi[k] \leftrightarrow G[\ulcorner \psi \urcorner, k])).$$

By Quantifier Distribution, since n is constant with respect to k ,

$$\vdash (\forall k) (\Box_{\mathcal{O}(\lg(k))}(\psi[k] \leftrightarrow G[\ulcorner \psi \urcorner, k])),$$

in which we can specialize to the forward implication,

$$\vdash (\forall k) (\Box_{\mathcal{O}(\lg(k))}(\psi[k] \rightarrow G[\ulcorner \psi \urcorner, k])).$$

By Implication Distribution of $\Box_{\mathcal{O}(\lg(k))}$,

$$\vdash (\forall k) (\forall a) (\Box_a(\psi[k]) \rightarrow \Box_{a+\mathcal{O}(\lg(k))}(G[\ulcorner \psi \urcorner, k])).$$

By Implication Distribution again, this time of $\Box_{a+\mathcal{O}(\lg(k))}$ over the implication $G[\ulcorner \psi \urcorner, k] = \Box_{g(k)}(\phi[k]) \rightarrow p[k]$, we obtain

$$\vdash (\forall k) (\forall a) (\forall b) \left(\Box_a(\psi[k]) \rightarrow \left(\Box_b(\Box_{g(k)}(\psi[k])) \rightarrow \Box_{a+b+\mathcal{O}(\lg(k))}(p[k]) \right) \right).$$

Now we specialize this equation to $a = g(k)$ and $b = h(k)$, where $h : \mathbb{N} \rightarrow \mathbb{N}$ is a computable function satisfying $e(g(k)) \prec h(k) \prec f(k)$, for example $h(k) = \lfloor \sqrt{f(k)e(g(k))} \rfloor$:

$$\vdash (\forall k) \left(\Box_{g(k)} \psi[k] \rightarrow \left(\Box_{h(k)} \left(\Box_{g(k)} (\psi[k]) \right) \rightarrow \Box_{g(k)+h(k)+\mathcal{O}(\lg(k))} (p[k]) \right) \right)$$

Then since $g(k) + h(k) + \mathcal{O}(\lg(k)) < f(k)$ after some bound $k > k_1$, we have

$$\vdash (\forall k > k_1) \left(\Box_{g(k)} (\psi[k]) \rightarrow \left(\Box_{h(k)} \left(\Box_{g(k)} (\psi[k]) \right) \rightarrow \Box_{f(k)} (p[k]) \right) \right)$$

Now, by hypothesis, $\vdash (\forall k) \left(\Box_{f(k)} (p[k]) \rightarrow p[k] \right)$, thus

$$\vdash (\forall k > k_1) \left(\Box_{g(k)} (\psi[k]) \rightarrow \left(\Box_{h(k)} \left(\Box_{g(k)} (\psi[k]) \right) \rightarrow p[k] \right) \right) \quad (4.4)$$

Also, without any of the above, from Bounded Inner Necessitation we can write

$$\vdash (\forall k) (\forall a) \left(\Box_a (\psi[k]) \rightarrow \Box_{e(a)} \left(\Box_a (\psi[k]) \right) \right).$$

From this, with $a = g(k)$, we have

$$\vdash (\forall k) \left(\Box_{g(k)} (\psi[k]) \rightarrow \Box_{e(g(k))} \left(\Box_{g(k)} (\psi[k]) \right) \right)$$

Now, since $e(g(k)) < h(k)$ after some bound $k > k_2$, we have

$$\vdash (\forall k > k_2) \left(\Box_{g(k)} (\psi[k]) \rightarrow \Box_{h(k)} \left(\Box_{g(k)} (\psi[k]) \right) \right) \quad (4.5)$$

Next, from Equations 4.4 and 4.5, assuming we chose $k_2 \geq k_1$ for convenience, we have

$$\vdash (\forall k > k_2) \left(\Box_{g(k)} (\psi[k]) \rightarrow p[k] \right). \quad (4.6)$$

But from Equation 4.3, the implication here is equivalent to $\psi[k]$, so we have

$$\vdash_N (\forall k > k_2) (\psi[k]),$$

where N is the number of characters needed for the proof above. From this, by Bounded Necessitation, we have

$$\vdash \Box_N \left((\forall k > k_2) (\psi[k]) \right).$$

By Quantifier Distribution of $\Box_N$,

$$\vdash (\forall k > k_2) \left(\Box_{C+2N+\lg(k)} (\psi[k]) \right)$$

and since $C + 2N + \lg(k) < g(k)$ after some bound $k > \hat{k}$, taking $\hat{k} \geq k_2$ for convenience, we have

$$\vdash (\forall k > \hat{k}) \left(\Box_{g(k)} (\psi[k]) \right). \quad (4.7)$$

Finally, from Equations 4.6 and 4.7 we have, as needed,

$$\vdash (\forall k > \hat{k})(p[k]). \quad \dashv$$

4.1. Interpretation. Löb’s Theorem may be viewed as an obstacle to a formal system of logic “trusting itself” to soundly prove any statement p . Previously, one might have thought this obstacle was merely a quirk of infinities arising from the unbounded proof-existence predicate $\Box$. However, we see now that some bounded obstacle remains: namely, that a bounded logical system cannot trust itself “about moderately long proofs in general”. To see this interpretation, let $p[k]$ be any statement with a free parameter k , and $f(k) \succ e(\mathcal{O}(\lg(k)))$ be any function, representing “moderate largeness”. Then the hypothesis $\vdash (\forall k) \left(\Box_{f(k)}(p[k]) \rightarrow p[k] \right)$ of Theorem 1 says that our logical system generally trusts its proofs about $p[k]$, even if they are moderately long. However, this will imply that $\vdash \forall k > \hat{k}, p[k]$, which is bad news if $p[k]$ is sometimes false.

4.2. Making $f(k)$ small. The statement of Theorem 1 becomes stronger as one makes the function $f(k)$ smaller, but it must remain $\succ e(\mathcal{O}(\lg(k)))$ for the theorem to apply. The obstruction to making $f(k)$ small is hence the size of the proof expansion function e , which in real world software for writing proofs about proofs will be under some design pressure to be made small, to manage computational resources.

How small can e be made in practice? Gödel numberings for sequences of integers can be achieved in $\mathcal{O}(n)$ space [Tsai, Chang, and Chen, 2002] (where n is the length of a standard binary encoding of the sequence), as can Gödel numberings of term algebras [Tarau, 2013]. To check that one line is an application of Modus Ponens from previous lines, if the proof encoding indexes the implication to which MP is applied, is a test for string equality that is linear in the length of the lines. Finally, to check that an abbreviation has been applied or expanded, if the proof encoding indexes where the abbreviation occurs, is also a linear time test for string equality.

Thus, one can straightforwardly achieve $e(k) \in \mathcal{O}(k)$ for real-world theorem-provers. In that case, the condition $f(k) \succ e(\mathcal{O}(\lg(k)))$ amounts only to saying that $f(k) \succ \lg(k)$.

§5. Robust cooperation of bounded proof-based agents in the Prisoner’s Dilemma. Bárász 2014, LaVictoire 2014, and others have exhibited various agent-like logical formulae who can be viewed as playing the Prisoner’s Dilemma by basing their “decisions” on proofs about each others’ definitions (as strings). In particular, they proffer proof of the opponent’s cooperation as an unexploitable condition for cooperation. However, their “agents” are purely mathematical entities who decide whether to cooperate based on undecidable logical conditions. This leaves open the question of whether their results are achievable by real software with bounded computational resources.

So, consider the following program, where $G : \mathbb{N} \rightarrow \mathbb{N}$ is a function to be specified later:


```

def FairBot_k(Opponent) :
  let B = k + G(LengthOf(Opponent))
  search for a proof of length at most B that
    Opponent(FairBot_k) = Cooperate
  if found,
    return Cooperate
  else
    return Defect

```

In this program, the subroutine “search for a proof of length at most B that $(\dots)$ ” is defined as a process which searches, in lexicographic order, through all strings of length $\leq B$, checking each string for whether it is a proof of $(\dots)$. If any string turns up that is a proof of $(\dots)$, the search halts and sets **found** = **true**. If no such proof exists, the search continues until the (finite) set of strings of length $\leq B$ have been exhausted, then halts and sets **found** = **false**. In what follows, all that matters is the functional behavior of the proof search procedure: that it sets **found** = **true** if a proof of length $\leq B$ exists, and **found** = **false** otherwise.

The program FairBot_k may be viewed as a kind of proof-based “agent” that plays the Prisoner’s Dilemma in the following sense. Given any string, **Opponent**, representing the source code of another program, we can compute the pair¹

$$R(\text{FairBot}_k, \text{Opp}) := (\text{FairBot}_k(\text{Opp}), \text{Opp}(\text{FairBot}_k)).$$

Viewed as such, FairBot_k has the desirable property that the outcome

$$R(\text{FairBot}_k, \text{Opp}) = (\text{Cooperate}, \text{Defect})$$

will never occur: if its opponent defects, then FairBot will find no proof of its opponent’s cooperation, so FairBot will itself defect. This property is called being *unexploitable*.

At this point, a natural question arises: what happens when FairBot encounters a copy of itself? That is, what is $\text{FairBot}_k(\text{FairBot}_k)$? Each FairBot will be searching for a proof that the other will cooperate. As such, one might expect to see a bottomless regression that will exhaust the proof bound B . (“The first FairBot must prove that the second FairBot must prove that the first FairBot must prove that $\dots$.”) Thus, it seems like they will find no proof of cooperation, and hence defect.

However, this turns out not to be the case. Letting

$$p[k] := (\text{FairBot}_k(\text{FairBot}_k) = \text{Cooperate}),$$

it is a direct consequence of Theorem 1 that

$$\text{FairBot}_k(\text{FairBot}_k) = \text{Cooperate}$$

¹We must assume that Opp halts and returns either **Cooperate** or **Defect** in this case; note that FairBot_k always halts.

In fact a much stronger claim is true, as the next theorem will show. It will demonstrate a mutually cooperative program equilibrium (in the sense of Tennenholtz [2004]) among a wide variety of (unequal) agents, provided only that they employ a certain “principle of fairness”.

Given a non-negative increasing function G , we say that an agent (i.e. program) A_k taking a parameter $k \in \mathbb{N}$ is **G-fair** if for any opponent (i.e. program) Opp , we have

$$\vdash \Box_{k+G(\text{LengthOf}(\text{Opp}))} (\text{Opp}(A_k) = C) \rightarrow (A_k(\text{Opp}) = C),$$

where $\text{LengthOf}(\text{Opp})$ is the character length of the opponent’s source code.

In words, A_k is G -fair if finding a short proof of its opponent’s cooperation is a sufficient condition for A_k to cooperate, when “short” is flexibly defined to be an increasing function of its opponent’s complexity, i.e. $k + G(\text{LengthOf}(\text{Opp}))$. The agents FairBot_k defined above are G -fair (where G is the function appearing in line 2 of their source code), and the reader is encouraged to keep them in mind as a motivating example for the following result:

THEOREM 2 (Robust cooperation criterion). *Suppose that*

- $e(k)$, the proof expansion function of our proof system as defined in Section 4, satisfies $e(\mathcal{O}(\lg(k))) \prec k$, and
- $G(\ell)$ is any non-decreasing function satisfying $G(\ell) \succ e(\mathcal{O}(\ell))$.

Then, for any G -fair agents A_k and B_k , there exists some $r \gg 0$ such that for all $m, n > r$,

$$A_m(B_n) = B_n(A_m) = \text{Cooperate}$$

Feasibility of bounds in Theorem 2. Before the proof, recall from Section 4.2 that we can achieve $e(k) \in \mathcal{O}(k)$ for automatic proof systems that are designed for easy verifiability, in which case $e(\mathcal{O}(\lg(k))) \prec k$, as needed.

PROOF OF THEOREM 2. The proof will make use of Theorem 1 at the very end, but first requires some additional proof-theoretic analysis. For brevity we let

$$a(k) := G(\text{LengthOf}(A_k)), \quad (5.1)$$

$$b(k) := G(\text{LengthOf}(B_k)), \quad (5.2)$$

$$\alpha[m, n] := (A_m(B_n) = \text{Cooperate}), \text{ and} \quad (5.3)$$

$$\beta[n, m] := (B_n(A_m) = \text{Cooperate}) \quad (5.4)$$

so we can write the G -fairness conditions more compactly as

$$\vdash \Box_{m+b(n)} (\beta[n, m]) \rightarrow \alpha[m, n] \text{ and} \quad (5.5)$$

$$\vdash \Box_{n+a(m)} (\alpha[m, n]) \rightarrow \beta[n, m].$$

For later convenience, we also choose a non-decreasing computable function

$$f(k) \succ e(\mathcal{O}(\lg(k)))$$

such that

$$6f(2^\ell) \leq G(\ell).$$

For example, we could take $f(k) = \lfloor G(\lfloor \lg(k) \rfloor) / 6 \rfloor$.

Now, $\text{LengthOf}(A_k) > \lg(k)$ and $\text{LengthOf}(B_k) > \lg(k)$ since they reference the parameter k in their code. Applying G to both sides yields

$$a(k), b(k) > G(\lg(k)) \geq 6f(k). \quad (5.6)$$

Define an “eventual cooperation” formula:

$$p[k] := (\forall m > k)(\forall n > k)(\alpha[m, n] \text{ and } \beta[n, m])$$

Using Quantifier Distribution once on the definition of $p[k]$,

$$\vdash (\forall k) \left(\Box_{f(k)}(p[k]) \rightarrow (\forall m > k) \left(\Box_{C'} \left((\forall n > k)(\alpha[m, n] \text{ and } \beta[n, m]) \right) \right) \right)$$

where $C' = C + 2f(k) + \lg(m)$. Applying Quantifier Distribution again,

$$\vdash (\forall k) \left(\Box_{f(k)} p[k] \rightarrow (\forall m > k)(\forall n > k) \left(\Box_{C''} (\alpha[m, n] \text{ and } \beta[n, m]) \right) \right) \quad (5.7)$$

Where $C'' = 3C + 4f(k) + 2\lg(m) + \lg(n)$. Now, for m, n large and $> k$, we have

$$\begin{aligned} 3C + \lg(n) &< n && \text{and by (5.6),} \\ 4f(k) + 2\lg(m) &< 6f(m) < a(m). \end{aligned}$$

Adding these inequalities yields

$$3C + 4f(k) + 2\lg(m) + \lg(n) < n + a(m),$$

so for some k_1 , from (5.7) we derive

$$\vdash (\forall k > k_1) \left(\Box_{f(k)}(p[k]) \rightarrow (\forall m > k)(\forall n > k) \left(\Box_{n+a(m)}(\alpha[m, n]) \right) \right).$$

Similarly, we also have

$$\begin{aligned} 3C + 2\lg(m) &< m && \text{and} \\ 4f(k) + \lg(n) &< 5f(n) < b(n), && \text{so for some } k_2 \geq k_1, \end{aligned}$$

$$\begin{aligned} \vdash (\forall k > k_2) \left(\Box_{f(k)}(p[k]) \rightarrow \right. \\ \left. (\forall m > k)(\forall n > k) \left(\Box_{n+a(m)}(\alpha[m, n]) \text{ and } \Box_{m+b(n)}(\beta[n, m]) \right) \right). \quad (5.8) \end{aligned}$$

Thus by (5.5),

$$\vdash (\forall k > k_2) \left(\Box_{f(k)}(p[k]) \rightarrow (\forall m > k)(\forall n > k)(c(n, m) \text{ and } c(m, n)) \right), \text{ i.e.}$$

$$\vdash (\forall k > k_2) \left(\Box_{f(k)}(p[k]) \rightarrow p[k] \right).$$

Therefore, by Theorem 1 (and the note following it), for some $\hat{k}$ we have

$$\vdash (\forall k > \hat{k})(p[k]).$$

In other words, for all $m, n > \hat{k} + 1$,

$$A_m(B_n) = B_n(A_m) = \text{Cooperate.}$$

⊣

Theorem 2 interesting for four reasons:

1. **It is surprising.** Löb's theorem has not been applied much in the setting of game theory, and in fact 100% of the dozens of mathematicians and computer scientists that the author has asked to guess the output of FairBot_k (FairBot_k) have either guessed incorrectly (expecting the proof searches to enter an infinite regress and thus reach their bounds), or given an invalid argument for cooperation (such as “it would be better for the agents to cooperate, so they will”).
2. **It is advantageous.** When k is large, FairBot_k outperforms the classical Nash/correlated equilibrium solution (Defect, Defect) to the Prisoner's Dilemma when facing itself (or any other G -fair agent) in a one-shot game with no iteration and no future reputation.
3. **It is unexploitable.** That is, the outcome (Cooperate, Defect) will never occur with a FairBot as player 1. If an opponent will defect against FairBot , FairBot will find no proof of the opponent's cooperation, so FairBot will also defect.
4. **It is robust.** Previous examples of cooperative program equilibria studied by Tennenholtz [2004] and Fortnow [2009] all involved cooperation based on *equality of programs*, a very fragile condition. Such fragility is not desirable if we wish to build real-world cooperative systems. By contrast, the G -fairness criterion relies only on the provability of the opponent's cooperation, rather than details of its implementation, and therefore establishes mutual cooperation between a broad class of agents.

§6. Conclusion. Theorem 1 represents a resource-bounded generalization of Löb's Theorem, which can be applied to algorithms that read and write proofs using bounded computational resources, such as formal verification software. Theorem 2 makes use of Theorem 1, and some additional proof-theoretic analysis, to demonstrate how algorithmic agents who have access to one another's source codes can *inexploitably* achieve cooperative outcomes that out-perform classical Nash equilibria and correlated equilibria. Moreover, the condition for cooperation in Theorem 2, called “ G -fairness”, is more robust than previously known unexploitable cooperative conditions, which depended on literal source-code equality [Tennenholtz, 2004].

As a direction for potential future investigation, it seems likely that other agents described in the purely logical (non-computable) setting of Bárász 2014 and LaVictoire et al. 2014 will likely have bounded, algorithmic analogs, and that many more general consequences of Löb's Theorem—perhaps all the theorems of Gödel–Löb provability logic—will have resource-bounded analogs as well.

Appendix A. String abbreviations in proofs. In section 2.2, we required that our proof system allow the definition and use of abbreviations, because real-world proof systems (such as MetaMath [Megill, 2007]) do this, and because it makes our analysis easier. Abbreviations are just string substitutions which must be replaced by their previously defined values when reading a proof. For the sake of concreteness, we illustrate this with an example.

Recall the axioms of Peano Arithmetic in first-order logic (FOL):

$$\begin{aligned}
 P1 : & (\forall x)(Sx \neq 0) \\
 P2 : & (\forall x)(\forall y)(Sx = Sy \rightarrow x = y) \\
 P3 : & (\forall x)(x + 0 = x) \\
 P4 : & (\forall x)(\forall y)(x + Sy = S(x + y)) \\
 P5 : & (\forall x)(x \cdot 0 = 0) \\
 P6 : & (\forall x)(\forall y)(x \cdot Sy = (x \cdot y) + x)
 \end{aligned}$$

along with, for every formula $A(x, \bar{y}) = A(x, y_1, \dots, y_k)$, the induction axiom

$$\text{Ind}(A) : (\forall y_1)(\forall y_2)(\dots)(\forall y_k) \left(\begin{aligned} & \left(A(0, \bar{y}) \text{ and } (\forall x)(A(x, \bar{y}) \rightarrow A(Sx, \bar{y})) \right) \rightarrow (\forall x)(A(x, \bar{y})) \end{aligned} \right)$$

Below is a proof of the associativity of addition, which does not use abbreviations. Some sections involving first-order logic are summarized in a single step, but this is not necessary. This proof uses the induction axiom for the formula $A(z) = ((x + y) + z = x + (y + z))$:

By P3:

$$(x + y) + 0 = x + y \quad (\text{A.1})$$

By P3:

$$(x + y) + 0 = x + (y + 0) \quad (\text{A.2})$$

By FOL (substitution):

$$(x + y) + z = x + (y + z) \rightarrow S((x + y) + z) = S(x + (y + z)) \quad (\text{A.3})$$

By P4:

$$S((x + y) + z) = (x + y) + Sz \quad (\text{A.4})$$

By FOL (substitution of A.4 in A.3):

$$(x + y) + z = x + (y + z) \rightarrow (x + y) + Sz = S(x + (y + z)) \quad (\text{A.5})$$

By P4:

$$S(x + (y + z)) = x + S(y + z) \quad (\text{A.6})$$

By P4:

$$S(y + z) = y + Sz \quad (\text{A.7})$$

By FOL (substitution of A.7 in A.6):

$$S(x + (y + z)) = x + (y + Sz) \quad (\text{A.8})$$

By FOL (substitution of A.8 in A.5):

$$(x + y) + z = x + (y + z) \rightarrow (x + y) + Sz = x + (y + Sz) \quad (\text{A.9})$$

By $\text{Ind}(A(z))$ from A.9:

$$(\forall x)(\forall y)(\forall z)((x + y) + z = x + (y + z)) \quad (\text{A.10})$$

Part of this proof can be made slightly shorter (in characters) using an abbreviation; see below. An abbreviation, as it is meant here, is nothing more than a literal string to be replaced by another string upon reading and checking the proof; it has no particular type or grammatical role in the proof language aside from that. If we chose to give more structure to the types of abbreviations that are allowed, we could make the checking of our proofs more efficient, and indeed, this is done in real-world proof systems like MetaMath [Megill, 2007]. This restriction is not needed for the proofs in this paper, however, so we allow abbreviations to be any literal string substitution for simplicity.

Here is essentially the same proof as above, using an abbreviation, “%HYP”, defined on the third line of the proof:

By P3:

$$(x + y) + 0 = x + y \quad (\text{A.11})$$

By P3:

$$(x + y) + 0 = x + (y + 0) \quad (\text{A.12})$$

Define abbreviation:

$$\%HYP == “(x + y) + z = x + (y + z)” \quad (\text{A.13})$$

By FOL (substitution):

$$\%HYP \rightarrow S((x + y) + z) = S(x + (y + z)) \quad (\text{A.14})$$

By P4:

$$S((x + y) + z) = (x + y) + Sz \quad (\text{A.15})$$

By FOL (substitution of A.15 in A.14):

$$\%HYP \rightarrow (x + y) + Sz = S(x + (y + z)) \quad (\text{A.16})$$

By P4:

$$S(x + (y + z)) = x + S(y + z) \quad (\text{A.17})$$

By P4:

$$S(y + z) = y + Sz \quad (\text{A.18})$$

By FOL (substitution of A.18 in A.17):

$$S(x + (y + z)) = x + (y + Sz) \quad (\text{A.19})$$

By FOL (substitution of A.19 in A.16):

$$\%HYP \rightarrow (x + y) + Sz = x + (y + Sz) \quad (\text{A.20})$$

By Ind(A(z)) from A.20:

$$(\forall x)(\forall y)(\forall z)((x + y) + z = x + (y + z)) \quad (\text{A.21})$$

REFERENCES

- MIHÁLY BÁRÁSZ, PATRICK LAVICTOIRE, PAUL F. CHRISTIANO ET AL. [2014], *Robust cooperation in the prisoner’s dilemma*, *arXiv preprint 1401.5577 [cs.GT]*.
 GEORGE BOOLOS [1993], *The logic of provability*, Cambridge University Press, New York.
 RENÉ CORI AND DANIEL LASCAR [2001], *Mathematical logic: A course with exercises*, vol. Part II, Oxford University Press, New York.

- LANCE FORTNOW [2009], *Program equilibria and discounted computation time*, **TARK '09: 12th conference on theoretical aspects of rationality and knowledge**, ACM Press, pp. 128–133.
- PATRICK LAVICTOIRE, BENJA FALLENSTEIN, ELIEZER YUDKOWSKY ET AL. [2014], *Program equilibrium in the prisoner's dilemma via Löb's Theorem*, **Multiagent interaction without prior coordination: Papers from the AAAI-14 workshop**, AAAI Publications.
- NORMAN MEGILL [2007], *Metamath, a computer language for pure mathematics*, Lulu Press, Morrisville, NC, USA.
- PAVEL PUDLÁK [1998], *The lengths of proofs*, **Handbook of proof theory** (Samuel R. Buss, editor), Studies in Logic and the Foundations of Mathematics, vol. 137, Elsevier, Amsterdam, pp. 547–637.
- PAUL TARAU [2013], *Bijective size-proportionate Gödel numberings for term algebras*, Unpublished manuscript.
- MOSHE TENNENHOLTZ [2004], *Program equilibrium*, **Games and Economic Behavior**, vol. 49, no. 2, pp. 363–373.
- SHI-CHUN TSAI, JEN-CHUN CHANG, AND RONG-JAYE CHEN [2002], *A space-efficient Gödel numbering with chinese remainder theorem*, **19th workshop on combinatorial mathematics and computation theory**, pp. 192–195.

MACHINE INTELLIGENCE RESEARCH INSTITUTE
 2030 ADDISON STREET, BERKELEY, CA, USA 94704
E-mail: critch@intelligence.org
URL: <http://intelligence.org/>